

Crossmark

RECEIVED

REVISED

PAPER

GSPyNetTree-O4: an event validation tool used in the fourth LIGO-Virgo-KAGRA observing run

Sofía Álvarez-López^{1,2,*}, Man Leong Chan³, Franz S. Herbst^{3,4}, Dhatri Raghunathan³, Airene Ahuja³, Annudesh Liyanage³, Julian Ding³, Alejandro Garcia-Varela⁵, Raymond Ng⁶, and Jess McIver³

¹ LIGO Laboratory, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

² Kavli Institute for Astrophysics and Space Research and Department of Physics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

³ Department of Physics and Astronomy, University of British Columbia, Vancouver, British Columbia, V6T1Z4, Canada

⁴ Department of Physics, University of Konstanz, 78457, Konstanz, Germany

⁵ Departamento de Física, Universidad de los Andes, 111711, Bogotá, Colombia

⁶ Department of Computer Science, University of British Columbia, Vancouver, British Columbia, V6T1Z4, Canada

*Author to whom any correspondence should be addressed.

E-mail: sofiaal@mit.edu

Keywords: noise, glitch, gravitational waves, LIGO, Virgo, KAGRA, machine learning

Abstract

The frequent presence of non-Gaussian transient noise, or glitches, in gravitational-wave detector data can affect gravitational-wave searches, parameter estimation, and downstream analyses. To identify and mitigate transient noise near gravitational-wave candidates in a timely manner, the LIGO-Virgo-KAGRA Collaboration employs the Data Quality Report. In the fourth observing run, GSPyNetTree-O4 was deployed within this framework as a tool for glitch classification and event validation. We describe GSPyNetTree-O4 and the main developments relative to its predecessor, GSPyNetTree. The most important update was a new architecture that allowed the simultaneous identification of glitches and gravitational-wave signals when both were present in the same input. We also expanded and augmented the training set with examples in which simulated gravitational-wave signals overlapped with real glitches, and applied 60 Hz calibration corrections to better match the data expected during the fourth observing run. On test data, the low-mass, high-mass, and extremely high-mass classifiers identified 97.9%, 97.7%, and 95.4% of glitches, respectively. Among samples without a glitch, including gravitational-wave-only and No Glitch samples, the classifiers correctly reported no data-quality issues in 97.1%, 96.6%, and 96.0% of cases, respectively. We further assessed the robustness of GSPyNetTree-O4 on unseen glitch morphologies, a small set of Virgo glitches from the fourth observing run, and different choices of the Q -value used to construct the time-frequency inputs. GSPyNetTree-O4 was successfully deployed as a Data Quality Report tool and increased automation in gravitational-wave event validation workflows.

1 Introduction

Recent upgrades in instrumentation have steadily improved the sensitivity of ground-based gravitational-wave (GW) detectors, including LIGO [1, 2], Virgo [3, 4], and KAGRA [5, 6]. Up to the second part of the fourth observing run, the LIGO-Virgo-KAGRA Collaboration (LVK) has reported more than 250 GW signals [7–10], with additional candidates reported by external teams [11–14]. With analyses of the remaining fourth observing run (O4) data still ongoing¹, the catalog is expected to expand further. Future observing runs with increased detector sensitivity may also enable observations of GW sources that have so far remained undetected, such as core-collapse supernovae and spinning neutron stars.

However, GW detection is still challenging. The extreme sensitivity of ground-based interferometers makes these detectors prone to a variety of transient noise features, known as

¹<https://gracedb.ligo.org/superevents/public/O4/>

glitches, which can limit astrophysical analyses [15, 16]. These non-Gaussian, short-duration artifacts can arise from environmental and instrumental disturbances and often occur frequently [17]. For example, during the first part of O4 (O4a), a rate of approximately one glitch per minute² was observed in the LIGO detectors [18]. Such a high glitch rate increases the likelihood that GW candidates occur in close proximity to glitches. Indeed, this was the case for more than 30% of the GW candidates reported in the recent GWTC-4 catalog [10], as well as for the binary neutron star merger GW170817 [19]. Furthermore, it has been demonstrated across a broad range of compact-binary masses that glitches occurring close in time to GW signals can distort source property estimation [20–28].

To address these challenges, the LVK performs event validation to identify and mitigate glitches that could impact downstream analyses [15, 17, 18]. To manage the increasing rate of candidate signals, the LVK implemented the Data Quality Report (DQR) [29, 30] to automate the production of event validation results. The DQR compiles results from a variety of tools designed to assess the quality of detector data in the vicinity of a GW candidate. Despite the success of this approach in reducing validation time in previous observing runs [17], pre-existing tools had limitations that hindered further automation. For example, **GlitchFind** [31] can identify cases where glitches elevate detector noise relative to surrounding data stretches, but is less effective when the glitch rate is elevated for a longer period, e.g., during light scattering, which can last for hours when local ground motion is elevated [32]. Similarly, **iDQ** [33–35] is extremely effective in cases where one or more sensors witness a glitch, but is otherwise blind to glitches without witnesses, as is the case for most recurring short-duration glitches, e.g., blips [17, 18].

Álvarez-López *et al.* [36] previously introduced **GSpyNetTree**, a deep learning algorithm developed from the glitch classification framework **Gravity Spy** [37, 38], and demonstrated its potential to effectively mitigate these challenges and increase automation in the event validation process [36]. **GSpyNetTree** consisted of an ensemble of convolutional neural networks (CNNs) that classified inputs as one of several glitch classes, a GW signal, or No Glitch. Its performance, however, decreased when the background-noise properties differed from those represented in the training set, reflecting the limited variation in background noise within the training data. It also performed less reliably on glitch morphologies that were absent or poorly represented in the training set, including Low-frequency Lines, which became a prominent glitch class in O4 [18]. Other development pathways for **GSpyNetTree** have also been explored, including **GSpyNetTree-S** [39], a segmentation algorithm built on the **You Only Look Once** [40] architectures, and **GSpyNetTree-Bayes** [41], which incorporates Bayesian convolutional neural networks for uncertainty estimation.

In this paper, we describe **GSpyNetTree-O4**, which was deployed by the LVK at the beginning of O4 for glitch classification and event validation and operated throughout the run. It was implemented as part of the **dqr-tasks** package³ within the **DQRbuild** toolkit [30]. We also present the principal improvements relative to the original algorithm. In Section 2, we describe the methodology used to improve **GSpyNetTree-O4**’s performance relative to the original **GSpyNetTree**. In Section 3, we present performance results for the version of **GSpyNetTree-O4** that ran in production in the LVK’s DQR and enabled validation of events in the LVK’s most recently published catalogs [10, 42], as well as future LVK catalogs expected within the year. In Section 4, we describe pathways for future improvement toward full automation of GW candidate validation. In Section 5, we summarize our conclusions.

2 GSpyNetTree-O4 improvements

GSpyNetTree-O4 follows the architectural philosophy of its predecessor [36]. It consists of a decision tree of three InceptionV3 [43] CNNs, trained on simulated GW signals and morphologically similar glitches. Both **GSpyNetTree** and **GSpyNetTree-O4** use a feature set similar to **Gravity Spy** [37, 38, 44–46]: each input sample is composed of a 2×2 matrix of spectrogram visualizations that are 0.5, 1, 2, and 4 seconds in duration. To capture the different morphologies between short-duration and long-duration glitches and GW sources, as recommended by Jarov *et al.* [47], different models were deployed depending on a GW candidate’s estimated total mass, M : the Low-mass (LM) classifier handled events with $M < 50 M_{\odot}$, the High-mass (HM) classifier covered events in the $50 M_{\odot} \leq M < 250 M_{\odot}$ range, and the Extremely high-mass (EHM) classifier was used for events with $M \geq 250 M_{\odot}$.

For O4, **GSpyNetTree-O4** was deployed on LIGO data as part of the LVK event-validation

²Including glitches with a signal-to-noise ratio above 6.5.

³Publicly available at <https://pypi.org/project/dqrtasks/>

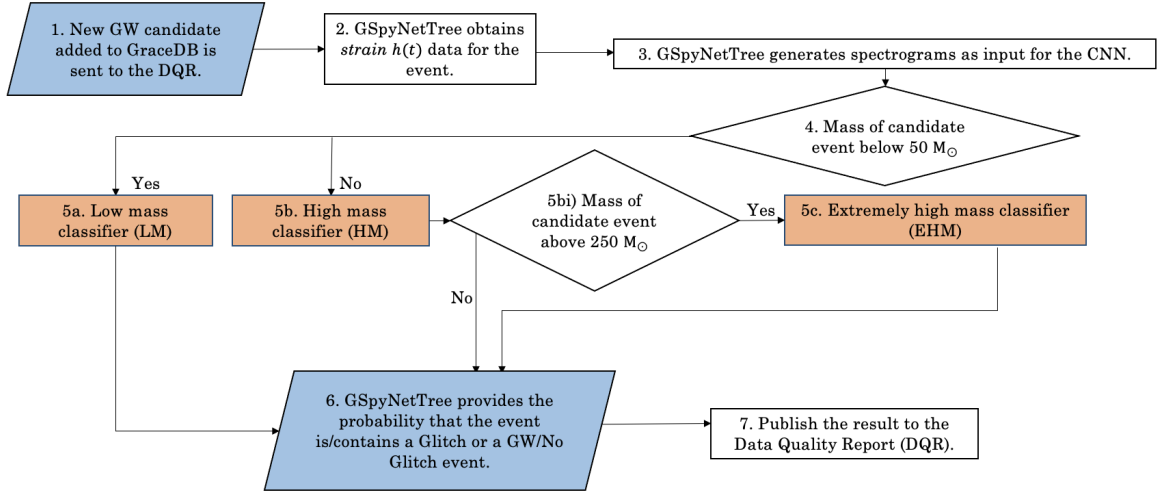


Figure 1: GSpyNetTree-04’s architecture (cf. Figure 3 in Álvarez-López *et al.* [36]). The addition of a new candidate to GraceDB [48] initiates the DQR workflow [30], which runs a suite of event-validation tasks, including GSpyNetTree-04. GSpyNetTree-04 obtains strain $h(t)$ data associated with the event and generates a 2×2 grid of spectrograms with durations of 0.5, 1, 2, and 4 seconds. These time-frequency visualizations are sent to one of three classifiers based on the candidate’s estimated total mass. Each classifier outputs probabilities indicating whether the event is or contains one of the target glitch classes, a GW, or no glitch. Finally, the result is published in the form of a webpage to the DQR, accessible to LVK event validators.

process [18]. When a new GW candidate was uploaded to GraceDB⁴ [48], the DQR triggered GSpyNetTree-04’s analysis [30]. GSpyNetTree-04 applied the appropriate model (LM, HM, or EHM) depending on the total mass of the candidate, which was estimated using the low-latency pipelines PyCBC Live [49, 50], MBTA [51, 52], GstLAL [53–56], or SPIIR [57, 58] and reported to GraceDB. The selected model then predicted the probability of the presence of a glitch in detector data surrounding the GW candidate. The results were published on a DQR webpage, following the architecture shown in Figure 1. GSpyNetTree-04’s output was used by LVK event validators to assess the quality of candidate events and inform the need for glitch mitigation, along with the output of other DQR tasks like GlitchFind and Omega Overlap [18, 30].

Building on GSpyNetTree’s original architecture, we developed GSpyNetTree-04 by implementing several key recommendations from Álvarez-López *et al.* [36]. These enhancements made GSpyNetTree-04 more robust to variations in background noise and GWs occurring in close proximity to glitches. In the following sections, we expand on the implementation of these improvements, along with other upgrades introduced in GSpyNetTree-04.

2.1 Construction of a representative and comprehensive glitch sample set

In Álvarez-López *et al.* [36], all glitches in the training set were obtained through LIGO-DV web [59], and the Gravitational Wave Open Science Center (GWOSC) [60–62]. In this work, we continued to use these glitches for GSpyNetTree-04, but we increased the number of glitch samples in the training set in two ways. First, we included glitches found in Virgo data from the third observing run (O3). Although GSpyNetTree-04 was deployed on LIGO data, Virgo glitches in the classes considered here exhibit morphologies similar to their LIGO counterparts. Including these samples allowed us to improve robustness to subtle morphological variations within the same glitch category. Second, we increased the number of glitches in our training set by including 193 manually verified glitches from non-quiet data segments flagged by the algorithm described in Section 2.2. We included only glitches with excess power above 10 Hz. Figure 2a shows an example of a Koi Fish glitch identified in this way during O3 at LIGO Livingston Observatory (LLO). We also found a considerable number of Virgo noise transients below 10 Hz, one of which is shown in Figure 2b. We did not include these low-frequency transients because GSpyNetTree-04 was developed primarily to run on LIGO data. A future version designed for Virgo data will require an expanded training set that includes these types of transients.

Table 1 shows the total number of glitch samples included for each class, as well as the glitch

⁴<https://gracedb.ligo.org>

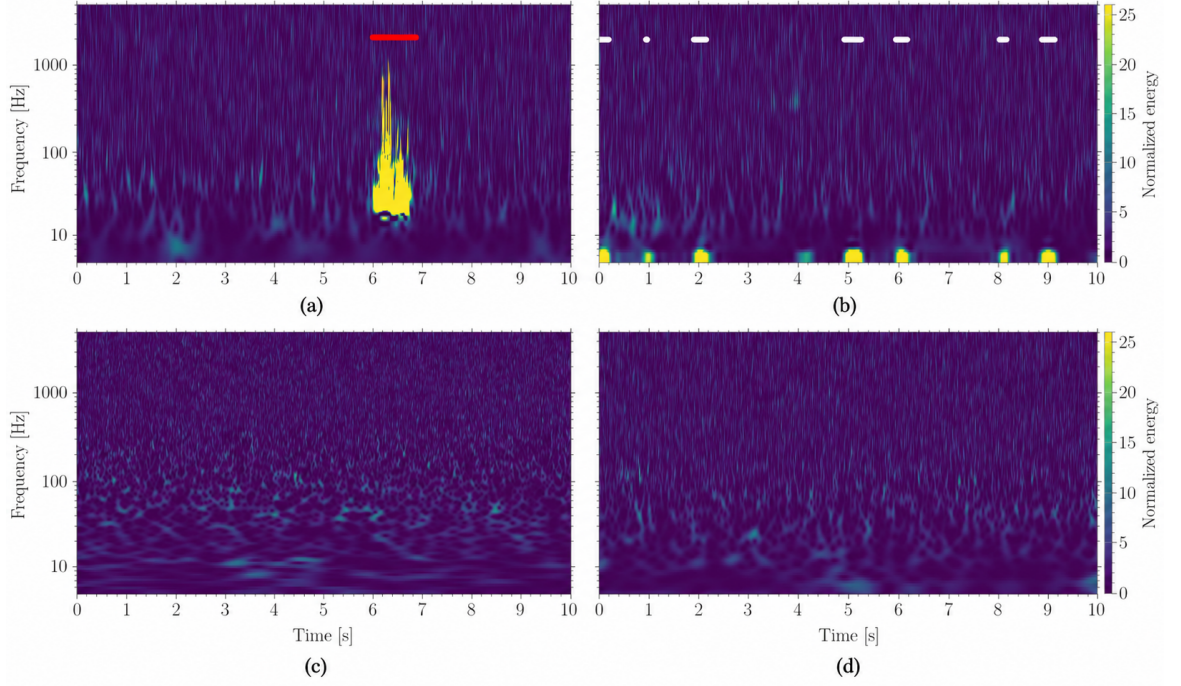


Figure 2: *Top panel:* Spectrograms of noisy detector data: (a) shows a Koi Fish glitch identified at LLO in O3. The time-frequency tiles flagged by the algorithm described in Section 2.2 are shown in red. This glitch was subsequently included in `GSpyNetTree-04`’s training set. (b) shows a Virgo spectrogram with abundant low-frequency glitches below 10 Hz. The time-frequency tiles flagged by the algorithm are shown in white. *Bottom panel:* Spectrograms of clean detector background times from (c) LIGO Hanford Observatory (LHO) and (d) LLO during O3. In `GSpyNetTree-04`, these clean times were used to inject simulated GW signals.

GSpyNetTree label	Total number of samples	Low-mass classifier	High-mass classifier	Extremely high-mass classifier
Blip	7725	✓	✓	✓
Low-frequency Blip	5000	✓	✓	✓
Fast Scattering	5305	✓	✓	✓
Light Scattering	4215	✓	✓	✓
Low-frequency Lines	5250	✓	✓	✓
No Glitch	14610	✓	✓	✓
Koi Fish	5330	✓	✓	
Scratchy	5305	✓		
Tomte	7085		✓	
GW (3 – 50 $M_{\odot}$)	14218	14218		
GW (50 – 250 $M_{\odot}$)	17489		17489	
GW (250 – 350 $M_{\odot}$)	12493			12493
Overlapping samples				
Blip + GW	36209	12070	12069	12070
Low-frequency Blip + GW	23880	7960	7960	7960
Fast Scattering + GW	37050	12350	12350	12350
Light Scattering + GW	35788	11928	11930	11930
Koi Fish + GW	9600	4800	4800	0
Scratchy + GW	5305	5305	0	0
Tomte + GW	11045	0	11045	0

Table 1: Number of samples per class for the LM, HM, and EHM classifiers. These total numbers include the following data augmentation techniques: 1) doubling the number of LIGO samples by applying the nonlinear 60 Hz AC power artifact subtraction for noise robustness (described in Section 2.3), and 2) applying time-offset augmentation, which adds four more examples by shifting the glitch or GW from $t = 0$ s.

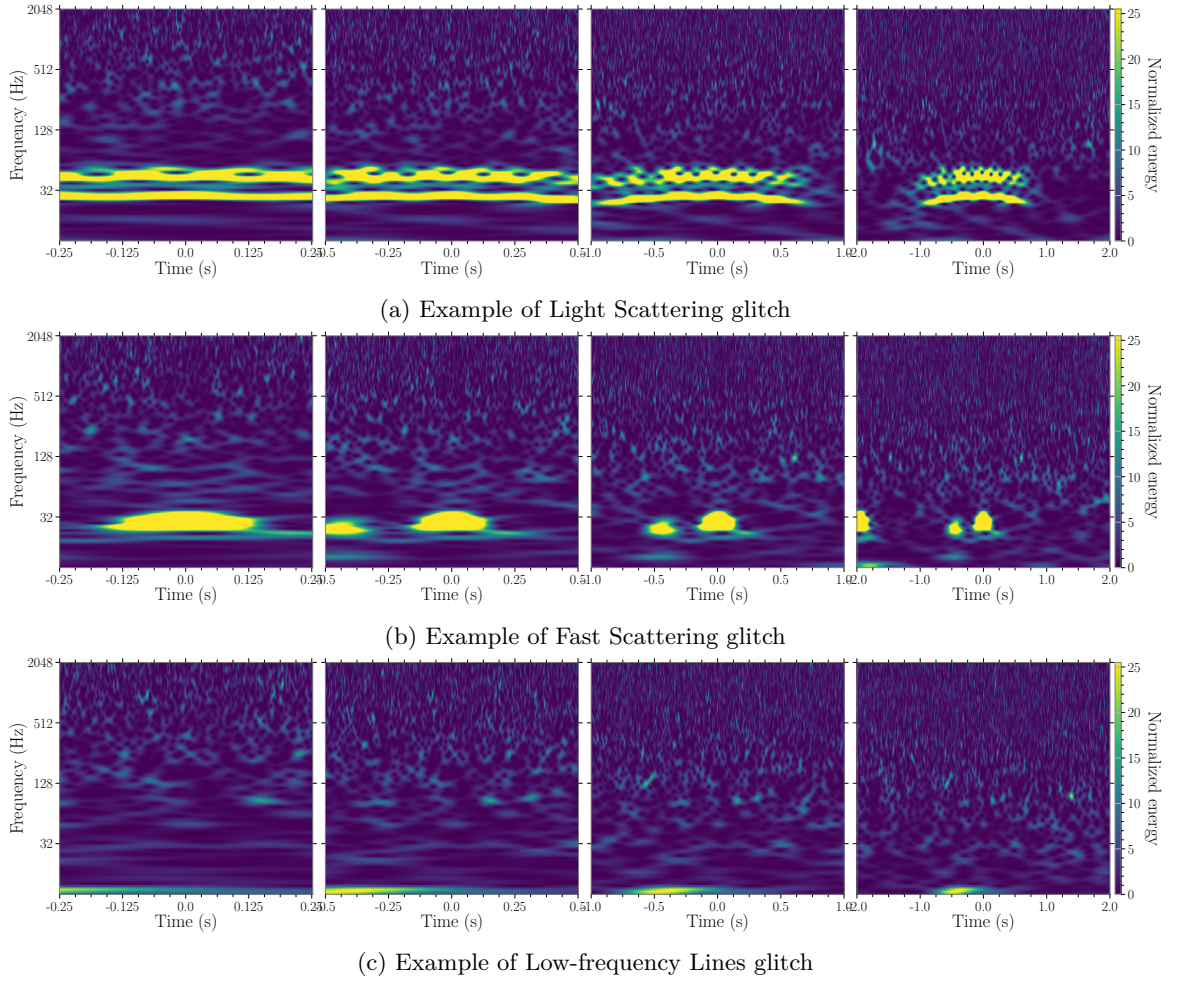


Figure 3: Examples of the new glitch classes incorporated in the data set for **GSpyNetTree-04**. The spectrograms are shown at four different durations (i.e., 0.5 s, 1 s, 2 s, and 4 s). We introduced these glitch classes as new classes to **GSpyNetTree-04** due to their frequent recurrence in O3 and ER15.

classes considered by each classifier in **GSpyNetTree-04**. In total, more than 80% of the samples came from LHO and LLO, while the remainder were from Virgo. Compared to **GSpyNetTree**, the LM classifier in **GSpyNetTree-04** included Koi Fish glitches. While Koi Fish are not morphologically similar to the GWs in this mass range, loud and saturated glitches can nevertheless overlap with GWs, as was the case of GW170817 [19]. Additionally, we included Light Scattering and Fast Scattering as new classes in all classifiers since they were the most common glitch types during O3 and were responsible for broad ranges of vetted times [63]. We also introduced Low-frequency Lines as a new class in all classifiers since they were the most common glitch class during Engineering Run 15 (ER15)⁵. Figure 3 shows examples of the three new glitch types included in **GSpyNetTree-04**.

2.2 Construction of a GW set in representative background noise

To generate GW samples, we simulated signals and injected them into quiet (i.e., glitch-free) time series detector data. For **GSpyNetTree**, all quiet segments were manually selected from a small set of manually inspected 64-second time series data segments from LHO and LLO. However, the selected time series segments only covered a few days during the first (O1) and second (O2) observing runs [36] and were not representative of the background noise of an entire observing run. Changes in detector configurations or environmental disturbances (e.g., earthquakes) may produce long-term variations in the background noise [18], which may be reflected in the spectrograms used as inputs to **GSpyNetTree-04**. In fact, Álvarez-López et al. [36] found that the performance of **GSpyNetTree** strongly depended on background noise.

⁵ER15 was a pre-observing phase prior to O4, which started on 2023-04-26.

Ensuring robustness to changes in background noise (i.e., steady-state, Gaussian detector noise) was a central focus of the O4 implementation of **GSpyNetTree**. For **GSpyNetTree-04**, we expanded the range of quiet time series segments used as background noise. In addition to the quiet data segments from O1 and O2 used in **GSpyNetTree**, we implemented a method that automatically identified quiet O3 data segments into which we could inject simulated GW signals. Our method for identifying quiet time series segments is similar to that of the **GlitchFind** DQR task [18, 30]. We calculated the energy of the time-frequency tiles of a spectrogram and verified Gaussianity by fitting an exponential distribution to the energy of the time-frequency tiles. If the normalized energy of four consecutive time-frequency bins exceeded 24, the time series segment corresponding to the spectrogram was flagged as a segment containing a glitch. This value was determined empirically. Otherwise, the time series segment was considered to be ‘quiet’ and was used for injecting GW signals.

Using this procedure, we obtained 984, 980, and 958 distinct ten-second quiet time series segments from O3 for LHO, LLO, and Virgo, respectively. This considerably increased the number of quiet time series segments available. While the development of **GSpyNetTree-04** was focused primarily on the implementation for LIGO detectors (**GSpyNetTree-04** did not run on Virgo data), including quiet Virgo time series segments made **GSpyNetTree-04** more robust to changes in background noise. Figures 2c and 2d show 10-second spectrograms of ‘quiet’ time identified by the algorithm from LHO and LLO, respectively.

After quiet time series segments were identified, we simulated and injected GW signals using a procedure similar to Álvarez-López et al. [36]. We first sampled parameters uniformly in the following ranges: total source-frame mass M between $5 M_\odot$ and $350 M_\odot$, with individual component source-frame masses m_1, m_2 ranging from $2 M_\odot$ to $175 M_\odot$; signal-to-noise ratio (SNR) between 8 and 35; individual dimensionless component spins a_1, a_2 between 0.05 and 0.95. We also randomized GW source orientations and sky locations across the entire sky. This was an improvement over the data set for **GSpyNetTree**, which assumed the optimal orientation of GW signals with respect to detectors. GW signals were then simulated using **LALSuite** [64] and the **IMRPhenomPv2** waveform model [65, 66]. In total, we simulated 1800, 2100, and 1500 GWs for the LM, HM, and the EHM classifiers, respectively, which were then injected into the time series obtained above.

To mitigate overfitting, we applied the data augmentation techniques suggested by Jarov et al. [47]. For each simulated GW signal, we generated four additional copies by randomly shifting the simulated GW signal in time. The time offsets were drawn from a uniform distribution over ± 0.5 s relative to the merger time. Having samples that were not always perfectly centered in the spectrograms made **GSpyNetTree-04** robust to small offsets in candidate merger times, as first reported in Álvarez-López et al. [36].

2.3 Construction of an overlapping set of GWs and glitches

With increased detector sensitivity and a higher detection rate, the probability of GWs overlapping with or occurring in close proximity to glitches increases. This scenario already occurred with more than 20% of candidates during O3 [8, 9], and more than 30% of candidates in the first catalog released from O4 data [10]. This issue has remained relevant throughout O4 [18].

As described in Álvarez-López et al. [36], **GSpyNetTree** followed a multi-class architecture. This meant that, given an input, the classifiers could only predict a single class, which was assumed to be independent and mutually exclusive from the others. When tested on overlapping samples of GWs and glitches, **GSpyNetTree** classified more than 60% (70% for the HM classifier) of GW signals as only glitches [36]. By identifying glitches as well as GWs in close proximity, **GSpyNetTree-04** could also inform event retraction and enable timely decisions for critical detector-characterization tasks like glitch mitigation [17, 18].

To enable simultaneous identification of both glitches and GWs present in the same input, we generated samples of overlapping GWs and glitches for training and testing **GSpyNetTree-04**. We also modified **GSpyNetTree-04**’s classifier architecture to enable the simultaneous identification of glitches and GWs. In this section, we discuss the generation of overlapping samples. The architecture changes made to the classifiers are discussed in the next section.

To test the performance of **GSpyNetTree** on samples where GW signals overlapped with a glitch, Álvarez-López et al. [36] generated 30 such samples for each glitch type. Each of them was created and manually verified by simulating 3 GW signals, and shifting them with respect to the glitch, using 10 time offsets drawn from a Normal distribution with mean $\mu = 0$ s and standard deviation $\sigma = 0.25$ s. With these parameters, a non-negligible number of glitches overlapped with the injected GWs. In some cases, this led to saturated spectrogram visualizations, where the GW

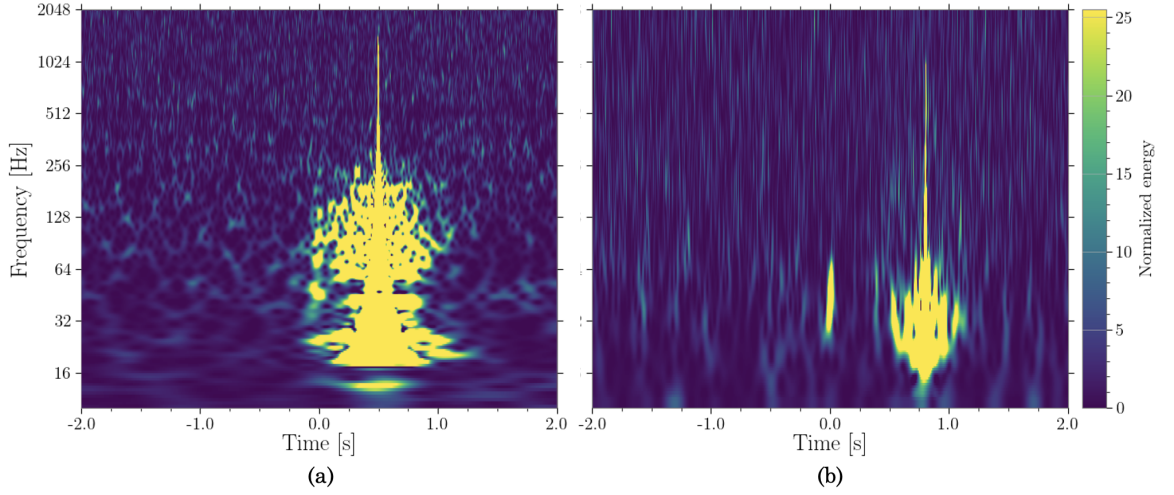


Figure 4: (a) Example spectrogram of a Koi Fish glitch that fully obscures the injected GW signal. The GW is unidentifiable due to the excess power dominating the time-frequency visualization. For reference, we are interested in generating spectrograms like panel (b): a glitch (right) occurs in close proximity to a GW signal (left), but without fully overlapping or obscuring it.

signal is completely obscured by the glitch. An example of this is shown in Figure 4a, where a Koi Fish glitch completely overlapped with a simulated GW signal. For reference, note that we were interested in generating examples like Figure 4b: GWs and glitches (Koi Fish, in this case) in close proximity but not fully overlapping. In Álvarez-López *et al.* [36], the few fully overlapping samples could be replaced manually. However, this process became unmanageable for a larger training set.

For **GSpyNetTree-04**, we designed a new method to inject GW signals near glitches. We selected 30 random examples of each glitch class and calculated their average duration, as shown in Table 2. We targeted the same common glitch classes as described in Álvarez-López *et al.* [36]. We found that glitches related to light scattering (e.g., Scratchy, Fast Scattering, and Light Scattering glitches [18, 32, 67]), were all long in duration (> 1.5 s). However, compared to GWs, they occurred at rather low frequencies (except for Scratchy glitches, which in turn had lower SNRs). In contrast, glitches related to a sharp, step-function-like change in the time series data (e.g., Blips, Low-frequency Blips, Tomte, and Koi Fish glitches [18, 68]) were shorter in duration (< 1 s) but could be found in a broader frequency range.

To generate samples of overlapping GWs and glitches, we first determined whether the glitch was short (Blip, Low-frequency Blip, Tomte, or Koi Fish) or long (Fast Scattering, Scratchy, or Light Scattering). If short, we generated two samples per GW signal: one sample with the glitch before the GW and one with the glitch after the GW signal. The offset in time between the GW signal and each glitch was drawn from a uniform distribution. For each sample, we ensured that the offset was large enough such that the glitches were not added directly on top of the GW signal. If the glitch was long, we generated three samples per GW signal: in addition to the two samples described above, we added one glitch exactly at the GW merger time. The reason we included this extra sample is twofold: first, it allowed us to represent in the data set the fact that long glitches like Light Scattering and Fast Scattering were the most common during O3 [17, 63]. Second, since they all occurred at low frequencies and/or had low SNR, the probability that the resulting spectrogram would be over-saturated was low.

To further minimize the probability of fully overlapping GWs and glitches, we standardized the Q -value of all our spectrograms to $Q = 20$. For some samples, fixing the Q -value at $Q = 20$ limited the energy per tile in our time-frequency visualizations, thereby lowering the likelihood of spectrogram saturation due to excess power. Additionally, it improved the visualization of low-mass GW signals, increasing the probability that these signals were correctly identified. In Section 4.2, we tested the impact of samples generated using different Q -values on the performance of the classifiers.

The lower section of Table 1 shows the samples of overlapping GWs and glitches for the LM, HM, and EHM classifiers, respectively. Because of the short time interval between ER15 (which started on 2023-04-26) and O4 (which started on 2023-05-24), we were unable to generate overlapping samples of GWs and Low-frequency Lines. The GW signals for all overlapping samples were generated with parameters drawn from the same ranges as described in Section 2.2.

Glitch type	Mean duration [s]
Blip	0.138
Low-frequency Blip	0.137
Tomte	0.346
Koi Fish	0.899
Fast Scattering	1.657
Scratchy	1.750
Light Scattering	2.073

Table 2: Average duration of the glitch types included in the training set for **GSpyNetTree-04**. Note that Blips, Low-frequency Blips, Tomte, and Koi Fish glitches are shorter in duration (< 1 s) than Fast Scattering, Scratchy, and Light Scattering glitches (> 1.6 s).

Additionally, in O4 the LIGO detectors implemented a nonlinear method to subtract alternating current (AC) 60 Hz power artifacts and harmonics, similar to the technique used for the O3 public data release [60]. Figure 5 in Álvarez-López *et al.* [36] shows a Blip glitch with (right panel) and without (left panel) the nonlinear subtraction applied. While the differences in both spectrograms may appear to be negligible, Álvarez-López *et al.* [36] showed that, when tested on glitches with the nonlinear subtraction applied, **GSpyNetTree**’s classification accuracy dropped from $> 90\%$ to $\sim 75\%$ for the three classifiers [36]. To better reflect the data that was expected in O4, we further augmented our data set by applying the nonlinear 60 Hz line-subtraction calibration to all our LHO and LLO glitches and GWs, including overlapping samples of GW and glitches. This increased robustness to a broader range of background noise toward O4. The total numbers of samples in our data set after the improvements and changes applied were 1.2×10^5 , 1.3×10^5 , and 1.0×10^5 for the LM, HM and EHM classifiers, respectively. For each classifier, most of the samples were from the LIGO detectors, while approximately 20% were from the Virgo detector. We used 80% of the entire data set for training (20% of which was used for validation). The remaining samples were used for testing.

2.4 Architecture

For **GSpyNetTree-04**’s three classifiers, we implemented a multi-label architecture, which allowed for the simultaneous identification of multiple classes within the same input [69]. We kept the InceptionV3 [43] model from **GSpyNetTree** due to the high accuracy achieved in Álvarez-López *et al.* [36]. To enable multi-label classification, we changed the output-layer activation function from **softmax** to **sigmoid** and optimized a binary cross-entropy loss for each output. To convert the output probabilities into predicted labels for evaluation, we treated a class as present when its probability exceeded 50%. We monitored recall rather than single-label accuracy during training. Recall is defined as the ratio of true positives to the sum of true positives and false negatives. A false negative occurs when **GSpyNetTree-04** fails to identify one or more objects present in the input; for example, when it detects no object in an input that contains one, or detects fewer than all objects in an input that contains multiple objects. By using recall as the evaluation metric, we directly quantified the fraction of glitches or GW signals that the classifier identified. The change from multi-class architecture to multi-label architecture was the most significant upgrade we included in O4, and it resulted in dramatic improvements in glitch identification in the presence of a GW candidate. We discuss our results in the next section.

3 Results

The primary objective of **GSpyNetTree-04** as a DQR task was to identify data quality (DQ) issues: glitches in the vicinity of a GW candidate. In this section, we evaluate the performance of **GSpyNetTree-04** on the test set, which consisted of 11960, 13216, and 9891 samples for the LM, HM, and EHM classifiers, respectively. Throughout the remainder of this paper, we used a threshold of 50% to determine whether **GSpyNetTree-04** predicted the presence of a given class in the input, consistent with the threshold used during training. Specifically, if the predicted probability for a class exceeded 50%, **GSpyNetTree-04** was considered to have identified that class. This enabled a direct comparison with the performance of **GSpyNetTree**. However, the DQR in O4 employed a more restrictive threshold for most statistical tasks, including **GSpyNetTree-04**, following the framework described in Davis *et al.* [30]. For **GSpyNetTree-04**, this meant requiring a high predicted glitch probability before issuing a DQ flag. This more restrictive criterion was

expected to reduce the false alarm rate by requiring higher-confidence glitch predictions, but it could also lower the true-positive rate by missing lower-confidence glitches.

To assess **GSPyNetTree-04**'s performance, we began by grouping all glitch classes under the same umbrella (including overlapping samples of GWs and glitches). This allowed us to determine whether a glitch was correctly flagged by **GSPyNetTree-04**. Figure 5 presents binary confusion matrices for the LM (left), HM (center), and EHM (right) classifiers, comparing the actual presence of glitches in the test data (vertical axis) with their predicted presence by **GSPyNetTree-04** (horizontal axis). In general, we aimed to maximize the true positive rate, shown in the bottom-right entry of each confusion matrix. The true-positive rate represents the fraction of glitches that were correctly identified by **GSPyNetTree-04**. We obtained true-positive rates of 97.9%, 97.7%, and 95.4% for the LM, HM, and EHM classifiers, respectively. These values corresponded to low false-negative rates for glitches, as shown by the bottom-left entries of the confusion matrices. While the EHM classifier had the lowest true-positive rate, it was also the least frequently used, as it was applied only when the candidate event's mass exceeded $250 M_{\odot}$. Through O4b, the most massive candidate detected by the LVK had a total mass of $M = 236^{+29}_{-48} M_{\odot}$ [10]. This event was handled by **GSPyNetTree-04**'s HM classifier.

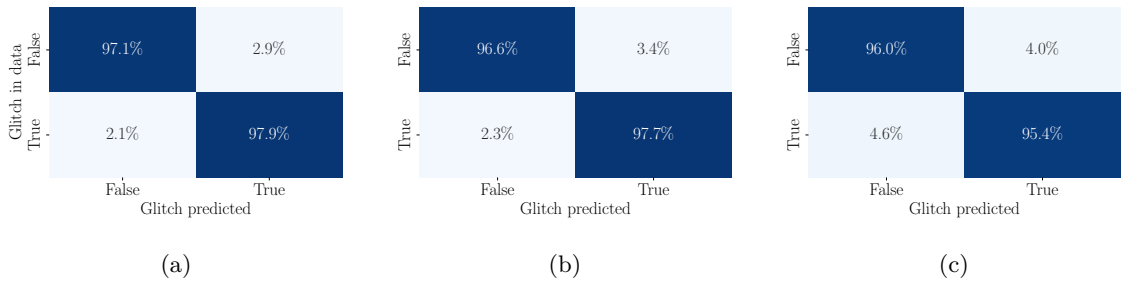


Figure 5: Glitch binary confusion matrices for the (a) LM, (b) HM, and (c) EHM classifiers on the test data. The bottom-right cell represents correct glitch identifications by **GSPyNetTree-04**. The classifiers identified 97.9%, 97.7%, and 95.4% of glitches, respectively. The bottom-left cell corresponds to the false-negative rate (i.e., the percentage of glitches that were not identified by **GSPyNetTree-04**), which is at most 4.6%. The top-right cell corresponds to the false-positive glitch rate (i.e., the percentage of samples in which a glitch was incorrectly predicted by the classifiers), which is at most 4.0%. Together, these matrices show high glitch-identification rates and low false-positive rates on the test sets.

We were also interested in achieving a low false-positive rate, shown in the top-right entry of each confusion matrix. A high false-positive rate would have meant that GW candidates were frequently and incorrectly flagged as having DQ issues. This would have triggered unnecessary additional event validation and reduced the trustworthiness of **GSPyNetTree-04**. The false-positive glitch rates were 2.9%, 3.4%, and 4.0% for the LM, HM, and EHM classifiers, respectively. Equivalently, the classifiers achieved high true-negative rates, shown in the top-left entries of the confusion matrices. This means that, among samples with no glitches present in the data, 97.1%, 96.6%, and 96.0%, respectively, were correctly classified as having no DQ issue.

To further understand the results of **GSPyNetTree-04** based on the labels on which it was trained, we grouped the predicted labels into four broad categories: GW, Glitch, both (GW+Glitch), or neither (No Glitch). We compared these to the true labels. The results are shown in Figure 6 as confusion matrices for each classifier. In general, the results of the three classifiers were very robust, consistent with the high rate of correct glitch identification shown in Figure 5.

The first row of each confusion matrix in Figure 6 shows that 73.4%, 81.3%, and 88.5% of the simulated GW signals were correctly classified by the LM, HM, and EHM classifiers, respectively. A substantial fraction of low-SNR signals were instead classified as No Glitch: 24.6%, 14.7%, and 9.3% for the LM, HM, and EHM classifiers, respectively. This effect was most pronounced for the LM classifier because low-SNR signals in this mass range have more extended time-frequency morphologies and therefore appear fainter than signals with the same SNR in the HM and EHM mass ranges. This makes them more difficult to distinguish from the No Glitch class. Ideally, in the presence of a real GW event, the GW label would be predicted instead of the No Glitch label. However, for event validation purposes, a GW classified as a No Glitch will not trigger further event validation. Combining the results of the GW and No Glitch prediction categories, we find that the LM, HM, and EHM classifiers correctly flagged no DQ issues in 98.0%, 96.0%, and 97.8%

True Label	GW	73.4%	1.0%	1.0%	24.6%
	Blip+GW	2.3%	87.8%	9.8%	0.2%
	Blip Low Frequency+GW	1.5%	93.5%	4.9%	0.0%
	Fast Scattering+GW	0.5%	90.4%	9.0%	0.1%
	GW+Koi Fish	0.0%	82.2%	17.8%	0.0%
	GW+Light Scattering	0.9%	88.5%	10.6%	0.1%
	GW+Scratchy	0.5%	56.9%	41.2%	1.4%
	Blip	0.0%	6.1%	92.3%	1.6%
	Blip Low Frequency	0.2%	5.1%	94.7%	0.0%
	Fast Scattering	0.0%	2.0%	98.0%	0.0%
	Koi Fish	0.0%	2.0%	98.0%	0.0%
	Light Scattering	0.0%	9.3%	90.4%	0.2%
	Low Frequency Lines	1.1%	0.2%	80.6%	18.1%
	Scratchy	0.0%	10.0%	88.4%	1.6%
	No Glitch	4.8%	0.1%	3.8%	91.3%
		GW	GW+Glitch	Glitch	No Glitch
Predicted Category					
(a)					
True Label	GW	81.3%	1.8%	2.3%	14.7%
	Blip+GW	2.3%	91.4%	5.8%	0.5%
	Blip Low Frequency+GW	3.5%	90.9%	5.3%	0.2%
	Fast Scattering+GW	1.1%	93.6%	5.3%	0.0%
	GW+Koi Fish	0.6%	90.6%	8.8%	0.0%
	GW+Light Scattering	1.2%	92.0%	6.7%	0.1%
	GW+Tomte	1.0%	93.0%	6.0%	0.0%
	Blip	0.0%	3.3%	96.7%	0.0%
	Blip Low Frequency	0.0%	0.8%	98.7%	0.4%
	Fast Scattering	0.2%	6.9%	89.4%	3.5%
	Koi Fish	0.0%	4.3%	95.7%	0.0%
	Light Scattering	0.0%	16.0%	82.2%	1.7%
	Low Frequency Lines	1.4%	0.2%	81.0%	17.3%
	Tomte	0.1%	7.5%	92.3%	0.0%
	No Glitch	8.2%	0.3%	2.2%	89.2%
		GW	GW+Glitch	Glitch	No Glitch
Predicted Category					
(b)					
True Label	GW	88.5%	0.4%	1.8%	9.3%
	Blip+GW	3.5%	87.2%	9.2%	0.2%
	Blip Low Frequency+GW	9.7%	83.8%	6.3%	0.2%
	Fast Scattering+GW	3.3%	90.2%	5.6%	0.8%
	GW+Light Scattering	1.9%	84.3%	13.7%	0.1%
	Blip	0.6%	3.2%	95.5%	0.7%
	Blip Low Frequency	2.2%	1.2%	94.5%	2.0%
	Fast Scattering	0.3%	2.7%	94.2%	2.7%
	Light Scattering	0.0%	6.3%	93.7%	0.0%
	Low Frequency Lines	0.2%	0.0%	85.7%	14.1%
	No Glitch	1.9%	0.1%	5.6%	92.5%
		GW	GW+Glitch	Glitch	No Glitch
Predicted Category					
(c)					

Figure 6: Specific confusion matrices for the (a) LM, (b) HM, and (c) EHM classifiers on the test data. The predicted labels on the x -axis are grouped into four categories: No Glitch, GW, Glitch (including all relevant categories), and GW+Glitch (samples in which both a GW and a glitch were predicted). The y -axis shows all true labels of the test samples. Each row is normalized to 100%. For example, 92.3% of Blips were classified by the LM classifier in the Glitch category.

of these samples, respectively. Equivalently, they falsely predicted a glitch in only 2.0%, 4.0%, and 2.2% of the cases.

We next consider the rows of Figure 6 corresponding to single-glitch test samples. More than 88%, 82%, and 93% of the samples from every glitch class except Low-frequency Lines were classified as Glitch only by the LM, HM, and EHM classifiers, respectively. Counting both Glitch and GW+Glitch predictions as successful glitch detections, these percentages increased to more than 98%, 96%, and 95%. Although some samples also received a spurious GW label, our main goal for **GSpyNetTree-04** was to identify the presence of glitches. These high glitch-identification rates highlight the model’s ability to flag DQ issues.

However, the fraction of correctly identified glitches was notably lower for the Low-frequency Lines class. Only 80.8%, 81.2%, and 85.7% of test samples from this class were correctly identified by the LM, HM, and EHM classifiers, respectively, while 18.1%, 17.3%, and 14.1% were misclassified as No Glitch. This is a consequence of the fact that we did not generate overlapping samples of Low-frequency Lines and GWs due to the short time between ER15 and O4. As a result, Low-frequency Lines were underrepresented in the training set. We expect the performance for this class to improve in future observing runs after adding Low-frequency Lines samples overlapping with GW signals. The lower fraction of correctly identified Low-frequency Lines samples, compared to the other glitch classes, highlights the importance of balanced and robust training sets in machine-learning applications.

3.1 Overlapping GWs and glitches

We now examine the rows of Figure 6 corresponding to samples containing overlapping glitches and GW signals. We first evaluated whether **GSpyNetTree-04** correctly identified both components

present in each sample: the glitch and the GW signal. The HM classifier performed best under this criterion: for all samples with overlapping GWs and glitches, **GSpyNetTree-04** correctly identified both components in more than 90% of the cases. The LM classifier correctly identified both components in $\geq 87\%$ of the samples, except for two scenarios: GW+Koi Fish (82.2%) and GW+Scratchy (56.9%). Since both Koi Fish and Scratchy span a broad frequency range and are typically very loud, GWs of moderate to low SNRs tend to be buried in their presence. Consistent with this interpretation, 17.8% of GW+Koi Fish samples were classified as Koi Fish only and 41.2% of GW+Scratchy samples were classified as Scratchy only.

Figure 7 shows two examples of Scratchy glitches overlapping with GW signals with total masses $M = 23.7 M_\odot$ (top panel) and $M = 10.9 M_\odot$ (bottom panel). In the first case, both glitch and GW were correctly identified, whereas in the second case only the glitch was detected. Because the primary purpose of **GSpyNetTree-04** was to flag the presence of glitches, its performance on these overlapping samples is more appropriately summarized by the glitch identification rate (i.e., the glitch recall). For GW+Koi Fish and GW+Scratchy samples, **GSpyNetTree-04** identified the glitch component in 100% and 98.1% of cases, respectively.

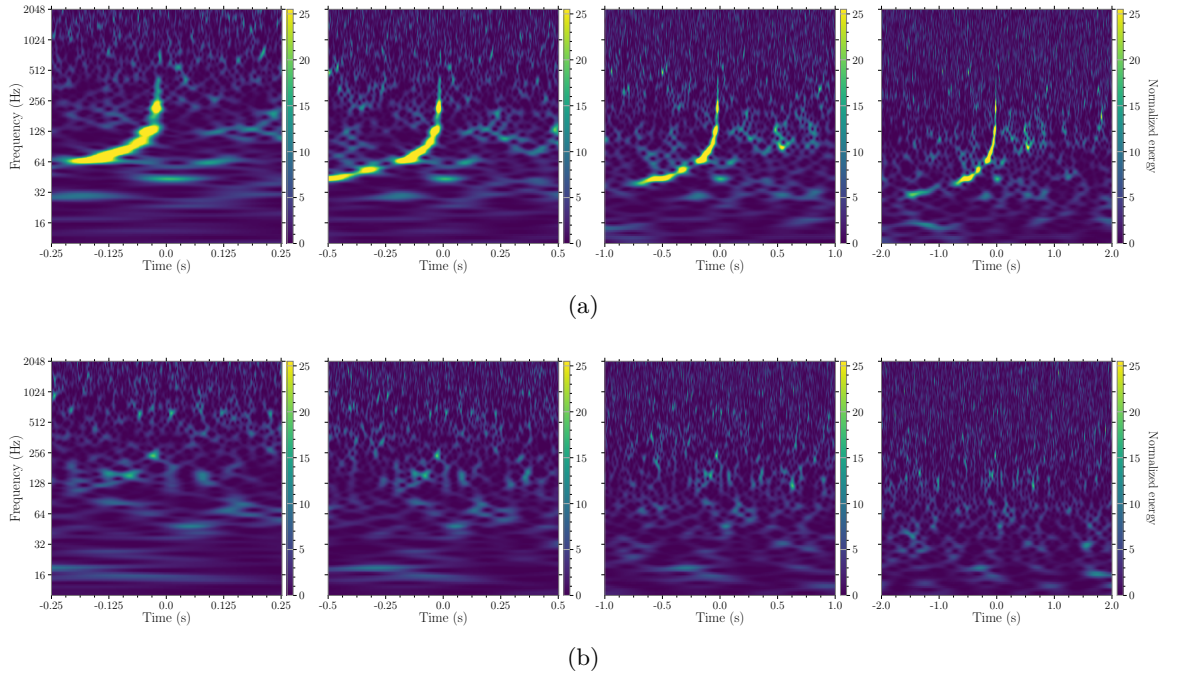


Figure 7: Samples of Scratchy glitches overlapping with GW signals with total masses (a) $M = 23.7 M_\odot$ and (b) $M = 10.9 M_\odot$. While the glitch is noticeable in both samples, the GW is only visible to the naked eye in the top spectrogram, since it has higher total mass and SNR. In fact, **GSpyNetTree-04** detected the glitch in both cases, but could only identify the GW signal in the top panel.

Due to its multi-label nature, **GSpyNetTree-04** could also predict more than two labels for a single input. Besides flagging a single glitch and a GW signal, it could also predict multiple glitch classes simultaneously. For instance, the time-frequency visualizations in Figure 8 show a Light Scattering glitch overlapping with a Low-frequency Line. **GSpyNetTree-04** successfully detected both glitches, demonstrating its robustness in flagging DQ issues.

To understand how both glitches were identified, we use **tf-explain** [70], a **TensorFlow** package to interpret neural network output. In particular, the Gradient-weighted Class Activation Mapping (**Grad-CAM**) module generates heatmaps that highlight regions of the 2×2 input spectrogram matrix contributing most to the HM classifier’s decision [71]. Figure 9a shows such a heatmap from an intermediate layer of our InceptionV3 architecture [43], while Figure 9b presents the one from its final convolutional layer, both computed for the sample in Figure 8. The yellow regions in the heatmap indicate areas of higher focus for the HM classifier. As seen in the bottom spectrograms of the 2×2 matrix in Figure 9a, the intermediate layers of the HM classifier focus on low-frequency regions, corresponding to the Low-frequency Line. As the input is processed through deeper layers of the network, the attention shifts to other regions, highlighting features associated

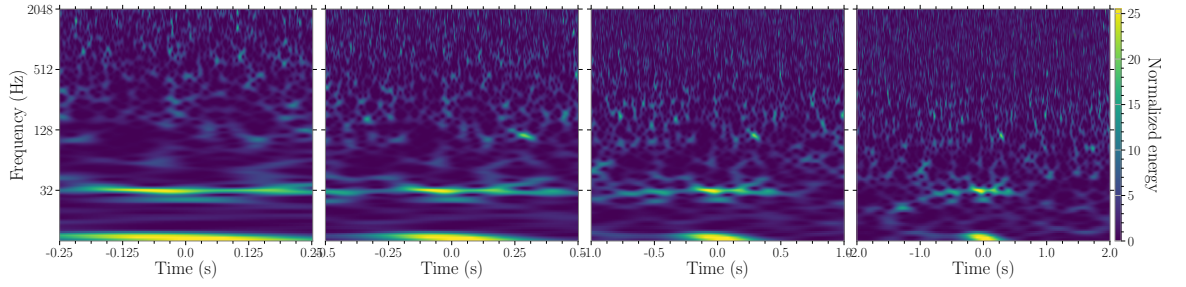


Figure 8: Spectrogram showing a sample in which **GSpyNetTree-04** predicts both a Light Scattering glitch at $f \approx 32$ Hz and a Low-frequency Line at $f \approx 12$ Hz.

with the Light Scattering glitch, as shown in Figure 9b. We observed this same behavior in other overlapping samples of GWs and glitches in the test set. This suggests that the network is learning to identify overlapping features at different depths of the network, gradually building up a more complete understanding of the input spectrograms, and highlighting the relevance of a robust architecture like InceptionV3.

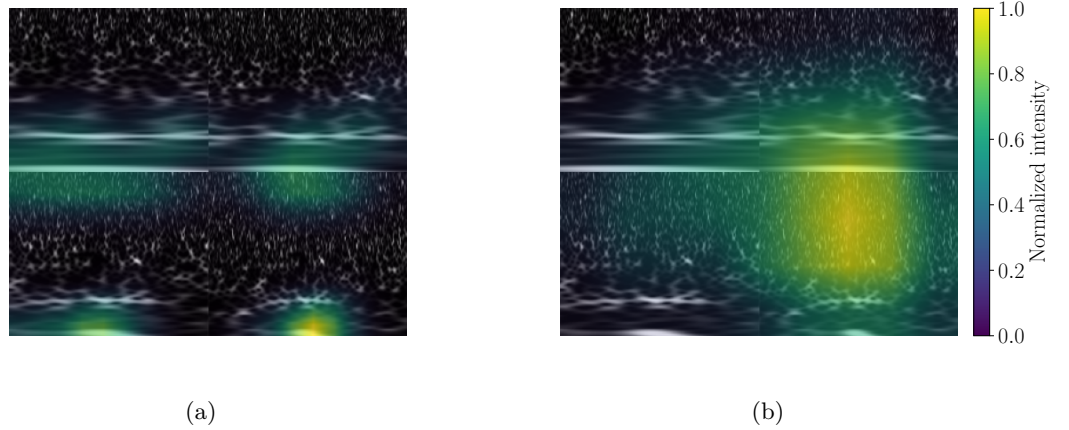


Figure 9: Heatmaps generated using the **Grad-CAM** [71] module of **tf-explain** [70] for (a) an intermediate and (b) final convolutional layer of the **GSpyNetTree** InceptionV3 architecture, applied to the sample in Figure 8. Brighter areas indicate higher relative intensity, on a scale from 0 to 1. It can be seen that in the intermediate layers, the CNN focuses on the Low-frequency Line in both spectrograms, while in the final convolutional layer, it focuses on broader features containing the Light Scattering glitch.

3.2 An example of **GSpyNetTree-04**'s performance on a retracted event

We use the retracted candidate S230708bi to illustrate how **GSpyNetTree-04** contributed to LVK event validation in practice. This candidate was identified in low latency during O4a on July 8 2023 with a false alarm rate of one per 0.44 years and an estimated total mass $\gtrsim 50 M_{\odot}$. It was subsequently circulated via NASA's Gamma-ray Coordinates Network [72] as a binary black hole merger candidate.

In parallel, the candidate triggered the HM classifier of **GSpyNetTree-04**, which correctly identified a Blip at the estimated merger time. The candidate was retracted approximately 40 minutes after its initial identification. **GSpyNetTree-04** contributed significantly to the data quality assessment that informed the retraction of this candidate [73]. In Figure 10, we show an excerpt of the summary page generated by **GSpyNetTree-04** in the DQR for this candidate. **GSpyNetTree-04** identified the glitch with a probability of 97.8%, flagging a DQ issue. The spectrograms used by **GSpyNetTree-04** for this candidate are shown in Figure 11.

4 Further studies on **GSpyNetTree-04** for future observing runs

We perform a series of validation studies to assess **GSpyNetTree-04**'s ability to generalize beyond the glitch morphologies represented in its training set. We suggest new avenues to improve glitch

*** GSPyNetTree results indicate a data quality issue. The p-value is 0.02143, which is below the threshold of 0.05. Contact other detchar experts to decide if any mitigation measures are required.

Start time (UTC)	2023-07-08 17:17:35.193934 (1372871873.193934)
End time (UTC)	2023-07-08 17:17:39.193934 (1372871877.193934)
Event ID	S230708bi
Window start (UTC)	2023-07-08 17:17:35.193934 (1372871873.193934)
Window end (UTC)	2023-07-08 17:17:39.193934 (1372871877.193934)
Channel	L1:GDS-CALIB_STRAIN
Estimated candidate mass	53.851685
Classifier used for training	High-mass
Predicted labels	Blip
Glitch p-value	0.02143

Class	Result	Probability
Blip		97.86%
Low_Frequency_Lines		2.30%
GW		0.20%
Fast_Scattering		0.10%
Light_Scattering		0.07%
Blip_Low_Frequency		0.05%
Tomte		0.02%
Koi_Fish		0.01%
No_Glitch		0.01%

13

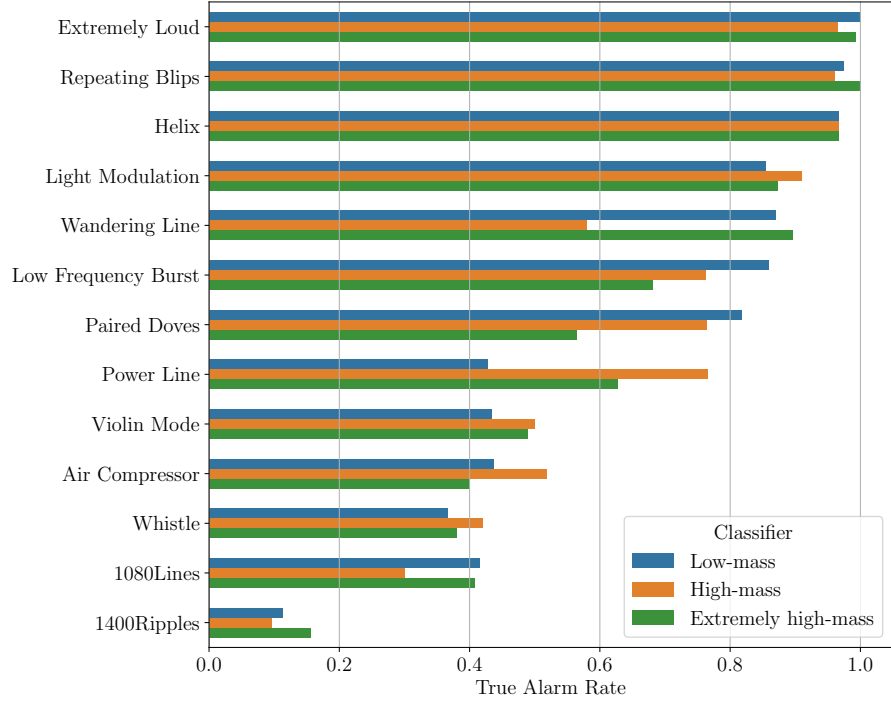


Figure 12: Probability that a sample from the **Gravity Spy** glitches not included in **GSPyNetTree-04**’s training set is classified as being/containing a glitch for the LM (blue), HM (orange), and EHM (green) classifiers.

identification as detector sensitivity and candidate rates increase. We start by testing **GSPyNetTree-04** on **Gravity Spy** glitch types we did not include in our training set as a proxy for evaluating **GSPyNetTree-04**’s performance on glitches with new and unknown morphologies. We then test the effect of Q -value for spectrogram generation on glitch and GW classification. Finally, we evaluate **GSPyNetTree-04**’s performance on a set of O4 Virgo glitches.

4.1 *GSPyNetTree-04*’s performance on unknown glitch types

We tested **GSPyNetTree-04**’s performance on all types of glitches classified by **Gravity Spy** [44] that we did not include in the data set described in Section 2. This test assessed whether **GSPyNetTree-04** could correctly classify inputs with morphologies different from those represented in the training set. To do this, we first selected approximately 30 glitches per class that were confidently classified by **Gravity Spy** (i.e., with **Gravity Spy** confidence greater than 90%). We then ran **GSPyNetTree-04** on these glitches. Figure 12 shows the true positive rate for each of the tested glitch types for all three classifiers. The true positive rate for a glitch type is defined as the fraction of tested samples of that glitch type that were correctly classified as glitches. For the Extremely Loud samples, the three classifiers achieved an average true positive rate $\geq 98\%$. This was expected since this glitch is morphologically similar to Koi Fish: both display saturated spectrograms, as shown in Figure 13a. We found a similar behavior with **GSPyNetTree**, but only for the HM classifier, which was the only one trained on Koi Fish glitches. The inclusion of Koi Fish glitches in the training set likely allowed **GSPyNetTree-04** to generalize to similar saturated glitches in the LIGO detectors across the three classifier mass ranges.

Additionally, for Repeating Blips (see Figure 13b), we obtained true alarm rates of 97%, 96%, and 100% for the LM, HM, and EHM classifiers, respectively. This represented a significant improvement over **GSPyNetTree**, where only $\sim 71\%$ (LM), $\sim 85\%$ (HM), and $\sim 53\%$ (EHM) of the Repeating Blips were classified as glitches. Many samples of Repeating Blips were classified by **GSPyNetTree-04** as a Blip (or other glitch) together with a non-existent GW. This suggested that the improved classification of Repeating Blips was partially a result of the multi-label architecture and the inclusion of overlapping samples in the training set. Together, these enabled **GSPyNetTree-04** to assign one instance of a blip as a Blip while assigning the others to different classes. Since in **GSPyNetTree-04** we were only interested in glitch identification, these classifications were acceptable. One potential solution is to deploy a segmentation model such as

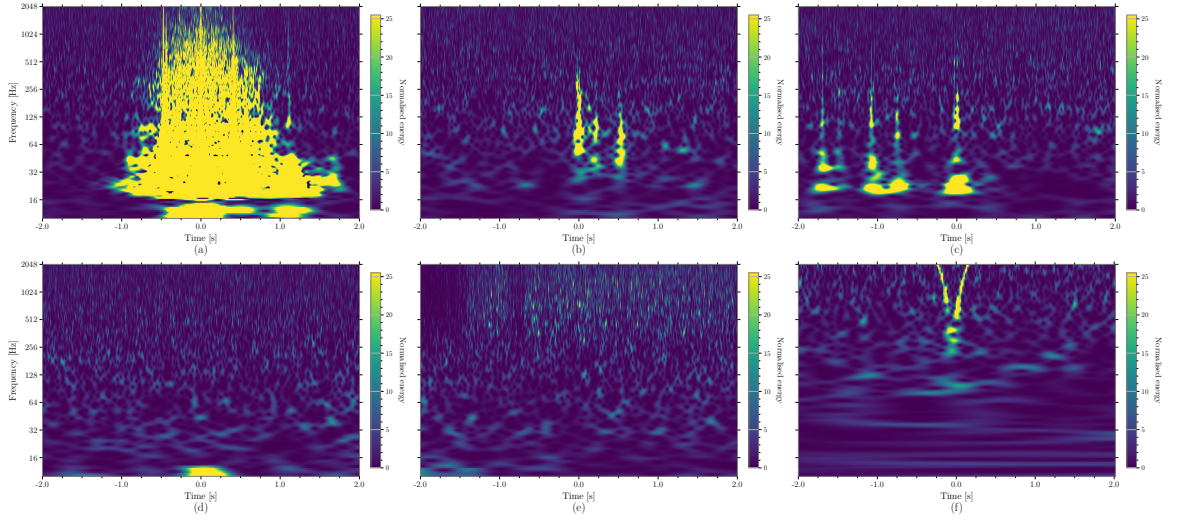


Figure 13: Time-frequency visualizations of some glitches not included in **GSPyNetTree-04**’s original training set, used for the validation study described in Section 4.1. The example glitches are: (a) Extremely Loud, (b) Repeating Blips, (c) Helix, (d) Low-frequency Burst, (e) Wandering Line, (f) Whistle.

GSPyNetTree-S [39], which can identify and localize multiple instances of one or more classes in a single input.

Moreover, **GSPyNetTree-04** correctly identified $\sim 97\%$ of Helix samples (Figure 13c) and $\sim 87\%$ of Light Modulation samples. These glitch types are morphologically similar to classes included in the **GSPyNetTree-04** training set. These results demonstrate **GSPyNetTree-04**’s ability to generalize predictions to similar, but new glitch morphologies. This was also the case for Low-frequency Bursts: in most cases, they were predicted as Low-frequency Lines. The identification of Low-frequency Bursts was lower than for Helix glitches due to the absence of Low-frequency Lines overlapping with GWs in the training set.

The Wandering Line glitches, shown in Figure 13e, closely resemble the Scratchy type. This morphological similarity may explain why the LM classifier identified more than 87% of these samples as glitches. Finally, we note that **GSPyNetTree-04** performed poorly on high-frequency glitch classes such as Whistles (Figure 13f), 1400 Hz Ripples, and 1080 Hz lines. Since **GSPyNetTree-04** was not trained on noise at such frequencies, it predicted the No Glitch class in more than half of the samples for all three classifiers. One solution to this would be to include a class for high-frequency glitches. Modifying **GSPyNetTree**’s architecture to predict excess power rather than GWs and specific glitch classes could also improve the identification of high-frequency noise. This direction has been explored for **GSPyNetTree-S** [39].

4.2 Effect of the spectrogram Q -value on **GSPyNetTree-04**’s classification performance

We studied the effect of the spectrogram Q -value on glitch and GW classification performance for **GSPyNetTree-04**. The Q -value determines the time-frequency resolution of the spectrogram: higher (lower) Q yields finer frequency (time) resolution [74]. Figure 14 shows the effect of the Q -value on the spectrogram representation of a Fast Scattering glitch for $Q \in [5, 10, 20, 30]$. Clearly, the time-frequency morphology of the glitch varies considerably with the Q -value (see Section 2 in Ferreira and González [75] for the impact of the Q -value in unsupervised machine learning methods).

To determine the optimal Q -value for each classifier, we regenerated the training set described in Section 2 with Q -values of 5, 10, 20, and 30, and retrained the LM and HM classifiers. We excluded Low-frequency Lines from the analysis, as Scattering glitches are similar to Low-frequency Line glitches in duration and frequency content. We therefore expected the impact of the Q -value on the time-frequency representation to be comparable. Additionally, we did not perform this analysis for the EHM classifier, as its glitch classes are a subset of those included in the HM classifier.

To assess **GSPyNetTree-04**’s performance in classifying samples for each Q -value, we built Receiver Operating Characteristic (ROC) curves for GWs and glitches in both the LM and HM classifiers. ROC curves illustrate the trade-off between True Alarm Rates (TARs) and False Alarm Rates (FARs) across various classification thresholds, which can be quantified by calculating the

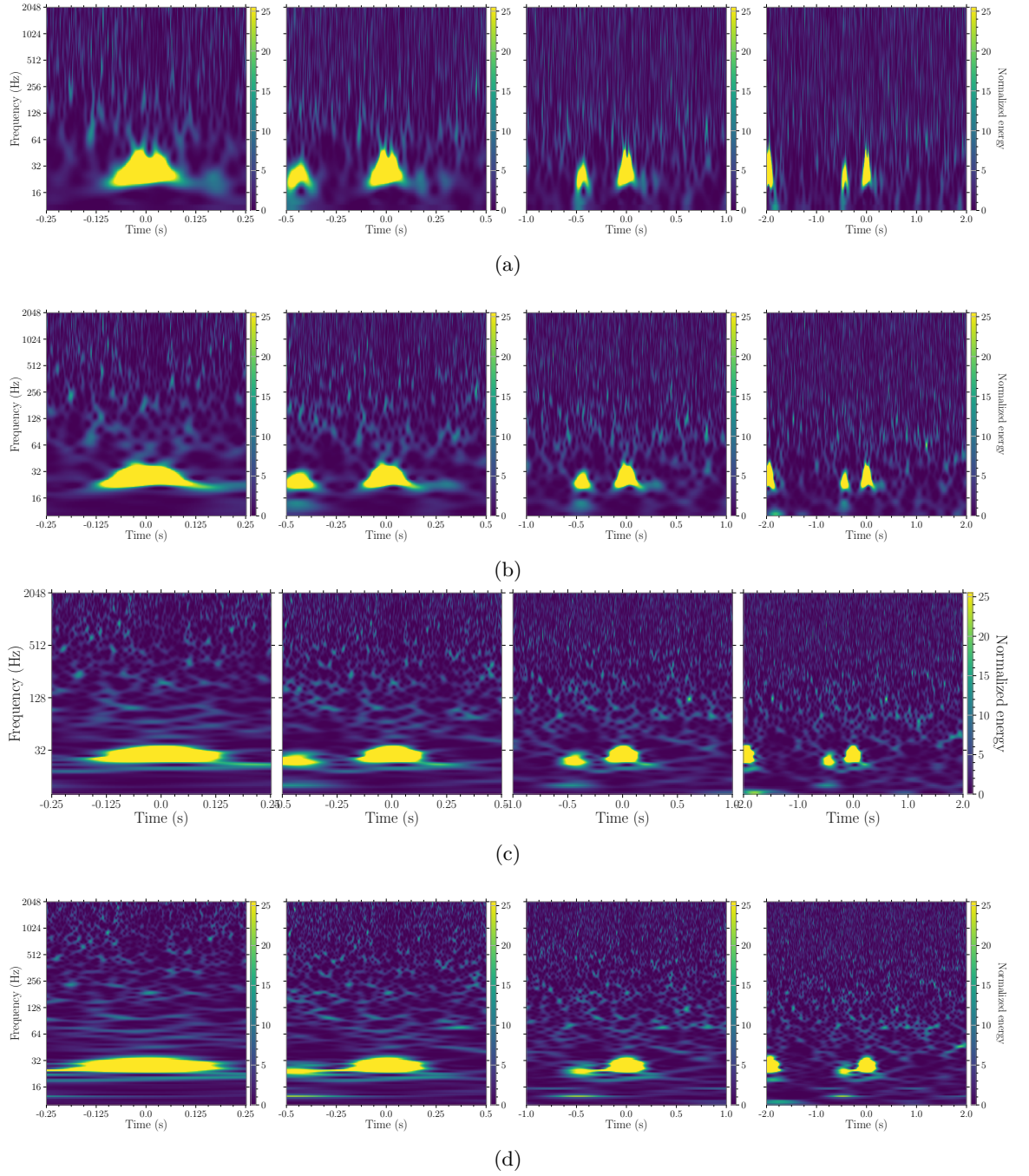


Figure 14: Fast Scattering glitch from LIGO Livingston at Q -values of (a) $Q = 5$, (b) $Q = 10$, (c) $Q = 20$, (d) $Q = 30$. Note that higher (lower) Q -values yield better frequency (time) resolution.

area under the curve (AUC). Larger AUC indicates better class discrimination, such that a perfect classifier would have an AUC of 1. Figure 15 shows an example of the ROC curves for the Blip glitch in the LM (left) and HM (right) classifiers, along with the calculated AUC, for the considered Q -values. Note that the AUCs are very close to 1 for all Q -values.

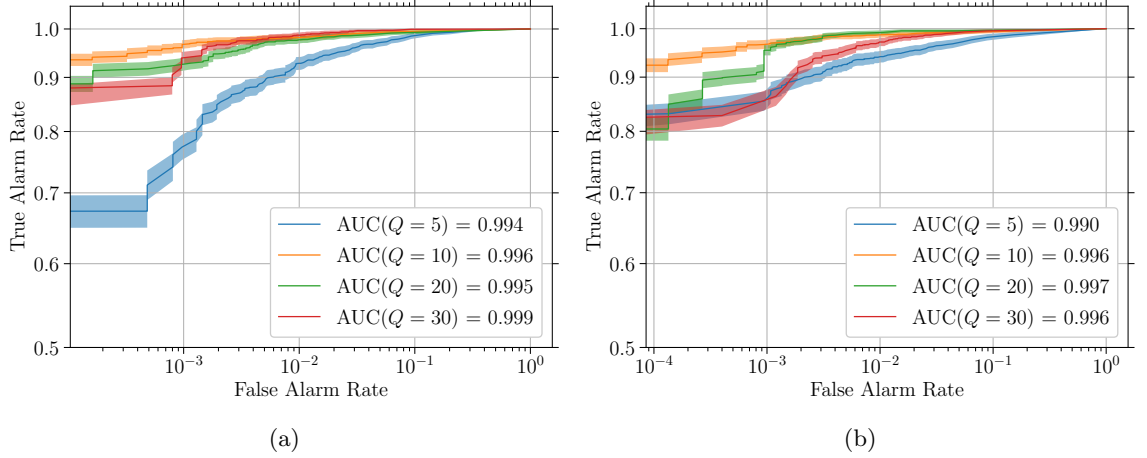


Figure 15: ROC curves for the Blip class in the (a) LM and (b) HM classifiers. The shaded regions represent pointwise $\pm 1\sigma$ binomial uncertainties on the True Alarm Rate (TAR) at each classification threshold. The AUC calculated for all Q -values is at least 0.99 in all cases, and the values differ only at the third decimal place. This shows the robustness of **GSpyNetTree-04** to different time-frequency resolutions.

Table 3 shows the performance, as measured by AUC, for all glitches and GWs at $Q \in [5, 10, 20, 30]$ in the LM (left) and HM (right) classifiers, respectively. Across glitch classes, AUC remains close to 1 for all tested Q -values, whereas the GW-class AUC is approximately 0.96 in both classifiers. This slightly lower GW performance is expected due to the similarity between low SNR signals and No Glitch. Therefore, to select the optimal Q -value for each classifier, we considered both the GW and mean AUC across all classes listed in Table 3.

For the LM classifier, the highest GW and average AUCs were achieved at $Q = 20$, which was the value used to generate **GSpyNetTree-04**’s data sets, described in Section 2. Therefore, this Q -value was the optimal for the LM classifier. In the HM classifier, the average AUC was the same at $Q = 20$ and $Q = 30$. However, the GW AUC was 0.3% higher at $Q = 30$, which made it the optimal Q -value. While the improvement in AUC at $Q = 30$ in the HM classifier should be considered for future observing runs, we kept $Q = 20$ as the default in O4 since it offered a good balance between time-frequency resolution across the three **GSpyNetTree** classifiers.

GSpyNetTree label	AUC (LM classifier)			
	$Q = 5$	$Q = 10$	$Q = 20$	$Q = 30$
Blip	0.994	0.996	0.995	0.999
Low-frequency Blip	0.996	0.999	0.997	0.992
Fast Scattering	0.984	0.991	0.995	0.994
Light Scattering	0.991	0.993	0.996	0.998
Koi Fish	1.000	0.999	1.000	1.000
Scratchy	0.988	1.000	0.996	1.000
No Glitch	0.961	0.962	0.982	0.951
GW (3 – 50 $M_\odot$)	0.944	0.936	0.949	0.947
Average	0.982	0.985	0.989	0.985

GSpyNetTree label	AUC (HM classifier)			
	$Q = 5$	$Q = 10$	$Q = 20$	$Q = 30$
Blip	0.990	0.996	0.997	0.996
Low-frequency Blip	0.991	0.992	0.996	0.989
Fast Scattering	0.994	0.991	0.996	0.995
Light Scattering	0.996	0.998	0.995	0.999
Koi Fish	0.993	0.990	0.997	0.998
Tomte	0.997	0.996	0.999	0.998
No Glitch	0.992	0.972	0.987	0.992
GW (50 – 250 $M_\odot$)	0.966	0.963	0.966	0.969
Average	0.990	0.987	0.992	0.992

Table 3: Performance of **GSpyNetTree-04**’s LM (left) and HM (right) classifiers, trained on spectrograms generated using different Q -values, as measured by area under the curve (AUC). The shaded cells show the highest GW AUC and average AUC for each classifier.

4.3 Testing **GSpyNetTree-04** on a set of Virgo glitches from O4

Finally, we evaluated **GSpyNetTree-04**’s performance on a set of 200 O4 Virgo glitches, selected from representative samples of the most prominent Virgo glitch classes in O4: “25 minute” glitches and Scattered Light. The samples were manually assigned to the closest **GSpyNetTree-04** morphology classes. Although this test was not intended to provide a comprehensive assessment of

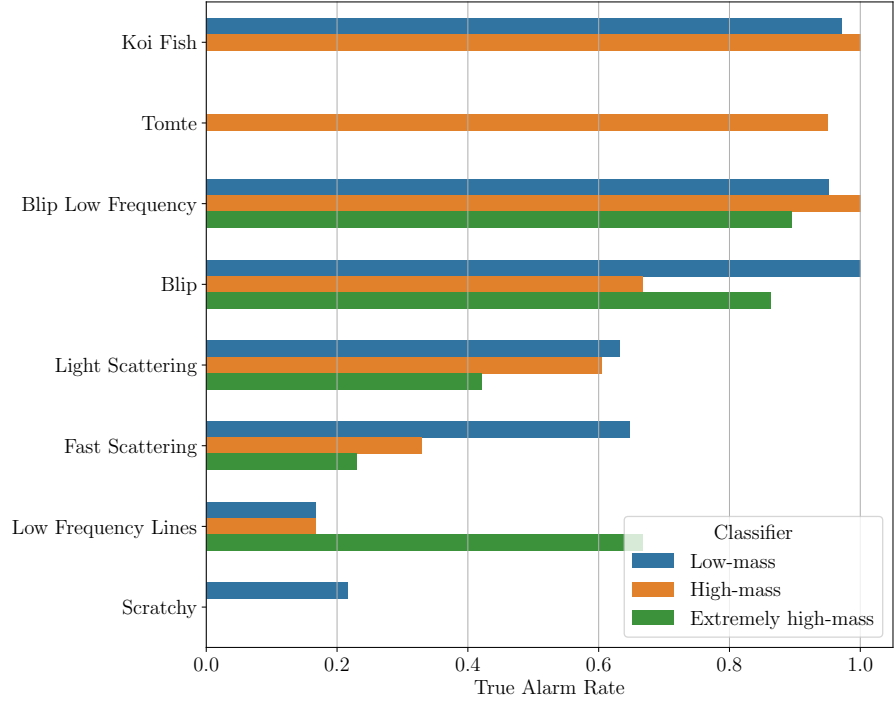


Figure 16: Fraction of samples from the Virgo glitch set classified as being/containing a glitch in the LM (blue), HM (orange), and EHM (green) classifiers. More than 90% of Koi Fish, Tomte, and Low-frequency Blip glitches are identified as glitches, while fewer than 60% of Light Scattering, Fast Scattering, Low-frequency Lines, and Scratchy glitches are identified as glitches.

GSPyNetTree-04's applicability to Virgo data, it served as a preliminary study of its transferability. As detailed in Section 2, **GSPyNetTree-04**'s design considerations were specifically targeted towards LIGO: most of our glitches and time series data came from LHO and LLO. In addition, we applied LIGO's O4 method of subtracting power artifacts and harmonics to the samples in our data set. However, our data set also contained Virgo data from O3. Therefore, we wanted to test whether **GSPyNetTree-04** could generalize to the most frequent Virgo glitches in O4, which have morphologies different from those of LIGO glitches.

Figure 16 shows the fraction of Virgo test samples classified as containing a glitch, regardless of whether a GW label was also predicted or whether the predicted glitch type was correctly identified by the classifiers. **GSPyNetTree-04** correctly detected a glitch in $\gtrsim 90\%$ of Koi Fish, Tomte, and Low-frequency Blip samples. For the Blip class, the LM classifier was able to identify all tested samples. However, the HM and EHM classifiers missed 33% and 14% of the tested samples. For all other classes (Light Scattering, Fast Scattering, Low-frequency Lines, and Scratchy), the glitch identification rate was $\lesssim 60\%$.

Overall, this performance was substantially below that obtained by **GSPyNetTree-04** on its test set, for which more than 95% of glitches were correctly detected (see Figure 5). Across the Virgo test set, more than 34%, 42%, and 51% of the samples analyzed by the LM, HM, and EHM classifiers, respectively, were incorrectly classified as containing no glitch. These false-negative rates are considerably higher than those shown in Figure 5 (2.1%, 2.3%, and 4.6% for each classifier, respectively).

These results suggest that a robust application to Virgo data would require a larger, more representative Virgo-specific training set. A representative Virgo training set may include samples of low-frequency noise, such as the one shown in Figure 2b. In Section 2.2, we found that such low-frequency noise was common in Virgo during O3, as first reported in Acernese et al. [4]. Future developments could consider building larger Virgo glitch data sets using Virgo-focused pipelines, such as **VIGILant** [76].

5 Conclusions

In this paper, we described **GSPyNetTree-04**, a machine learning algorithm for LVK event validation that was deployed in production as a DQR tool during O4. Building upon **GSPyNetTree**,

GSpyNetTree-04 consisted of an ensemble of three CNNs, referred to as the Low-mass (LM), High-mass (HM), and Extremely high-mass (EHM) classifiers, respectively. These classifiers, based on the InceptionV3 architecture [43], were designed to target GW candidates with total mass in different ranges as well as glitches morphologically similar to GWs in each corresponding mass range.

To develop **GSpyNetTree-04**, we implemented a series of modifications and improvements relative to **GSpyNetTree**, for both data generation and the classifier architecture. First, we expanded the set of background time series segments from LHO, LLO and Virgo by verifying the Gaussianity of the segments. This allowed us to not only add simulated GW signals in more representative background noise, but also to increase the number of glitches in our data set. Moreover, the most significant upgrade for data generation included the construction of a data set where glitches occurred in the proximity of GW signals. This data set was designed to mimic the increasingly likely scenarios where GW signals overlap with glitches as detector sensitivity increases. In addition, we applied calibration at 60 Hz to remove line artifacts, doubling the number of LHO and LLO samples in our data set. These improvements and modifications allowed us to construct a data set that was more representative of the data expected in O4. Finally, to enable simultaneous identification of GWs and glitches in a single input, we implemented a multi-label architecture for the LM, HM and EHM classifiers of **GSpyNetTree-04**, which was one of the most significant upgrades introduced in this work.

On the test sets, **GSpyNetTree-04** achieved glitch true-alarm rates $> 95\%$. Among samples with no true DQ issue, including GW-only and No Glitch samples, **GSpyNetTree-04** correctly reported no DQ issue in 97.1%, 96.6%, and 96.0% of cases for the LM, HM, and EHM classifiers, respectively. In the case of GWs, particularly in the LM regime, low-SNR signals were often misclassified as No Glitch. Likewise, when a low-SNR GW signal occurred near a glitch, **GSpyNetTree-04** typically detected only the glitch. Although this was sufficient for **GSpyNetTree-04**'s use case (i.e., identifying glitches responsible for or in the proximity of candidate events), in future work, we plan to develop improved classification techniques for fainter GW signals.

We further assessed **GSpyNetTree-04**'s ability to detect glitch types that were not included in the training set. We found that it generalized well to glitches that were morphologically similar to those in its training set, correctly identifying more than 95% of them as glitches. We also found similar performance when testing **GSpyNetTree-04** on glitches characterized by repeated excess power with morphologies resembling those in the training set (e.g., Repeating Blips). However, we found poor performance on high-frequency glitches, primarily because **GSpyNetTree-04**'s training set did not include these types of glitches.

We also evaluated **GSpyNetTree-04**'s performance on a small set of Virgo glitches from O4 as a preliminary study of its transferability to Virgo data in future observing runs. Because **GSpyNetTree-04** was developed primarily for LIGO data, most of the training samples came from the LIGO detectors. Additionally, the tested Virgo glitches were morphologically different from glitches in the training set. As a result, **GSpyNetTree-04** did not generalize well to the tested Virgo glitches. However, the development of a training set with sufficient and representative samples from Virgo is expected to improve the performance of **GSpyNetTree-04** for Virgo.

Moreover, we investigated the impact of different Q -values used for generating the training set for the LM and HM classifiers. We found that the Q -value employed in this work (i.e., 20) was the optimal for the LM classifier among the tested values, while the optimal value for the HM classifier was 30. Overall, the effect of the Q -value was small.

During O4, **GSpyNetTree-04** was successfully deployed as a DQR tool. It demonstrated promising classification performance and increased the degree of automation available to GW event validation workflows.

Acknowledgments

We thank Derek Davis and Heather Fong for useful discussions. The authors are grateful to Francesco DiRenzo for identifying the O4 Virgo glitches used in this paper and for serving as our internal LVK reviewer. SA-L. is supported by the Thomas A. Frank Fellowship Fund, the Santodomingo Fellowship Fund, and the ACM SIGHPC Data Science Fellowship. MC and JM are supported by an NSERC Discovery Grant, an NSERC Alliance award, CIFAR, and the McDonald Institute. JM is supported by a Tier II CRC. DR was supported by a Mitacs Globalink Research Internship award. AA was supported by an Erich Vogt First Year Summer Research Experience (FYSRE) award. JD, AL were supported by NSERC Undergraduate Summer Research Awards (USRAs). The authors are grateful for computational resources provided by the LIGO Laboratory supported by National Science Foundation Grants PHY-0757058 and PHY-0823459. This material

is based upon work supported by NSF’s LIGO Laboratory which is a major facility fully funded by the National Science Foundation.

References

- [1] LIGO Scientific Collaboration et al., “Advanced LIGO”, [Classical and Quantum Gravity](#) **32**, 074001 (2015).
- [2] E. Capote et al., “Advanced LIGO detector performance in the fourth observing run”, [Phys. Rev. D](#) **111**, 062002 (2025).
- [3] F. Acernese et al., “Advanced Virgo: a second-generation interferometric gravitational wave detector”, [Classical and Quantum Gravity](#) **32**, 024001 (2015).
- [4] F. Acernese et al., “Virgo detector characterization and data quality: results from the O3 run”, [Classical and Quantum Gravity](#) **40**, 185006 (2023).
- [5] T. Akutsu et al., “Overview of KAGRA: Detector design and construction history”, [Progress of Theoretical and Experimental Physics](#) **2021**, 05A101 (2020).
- [6] C. KAGRA, “Performance of the KAGRA detector during the first joint observation with GEO 600 (O3GK)”, [Progress of Theoretical and Experimental Physics](#) **2023**, 10A101 (2022).
- [7] B. P. Abbott et al. (LIGO Scientific Collaboration and Virgo Collaboration), “GWTC-1: A Gravitational-Wave Transient Catalog of Compact Binary Mergers Observed by LIGO and Virgo during the First and Second Observing Runs”, [10.1103/PhysRevX.9.031040](#).
- [8] B. P. Abbott et al. (LIGO Scientific Collaboration and Virgo Collaboration), “GWTC-2: Compact Binary Coalescences Observed by LIGO and Virgo During the First Half of the Third Observing Run”, [10.1103/PhysRevX.11.021053](#).
- [9] B. P. Abbott et al. (LIGO Scientific Collaboration and Virgo Collaboration and KAGRA Collaboration), “GWTC-3: Compact Binary Coalescences Observed by LIGO and Virgo During the Second Part of the Third Observing Run”, [10.48550/arXiv.2111.03606](#).
- [10] A. G. Abac et al., “GWTC-4.0: Updating the Gravitational-Wave Transient Catalog with Observations from the First Part of the Fourth LIGO-Virgo-KAGRA Observing Run”, (2025).
- [11] A. H. Nitz et al., “2-OGC: Open Gravitational-wave Catalog of Binary Mergers from Analysis of Public Advanced LIGO and Virgo Data”, [The Astrophysical Journal](#) **891**, [10.3847/1538-4357/ab733f](#) (2020).
- [12] A. H. Nitz et al., “3-OGC: Catalog of Gravitational Waves from Compact-binary Mergers”, [The Astrophysical Journal](#) **922**, [10.3847/1538-4357/ac1c03](#) (2021).
- [13] B. Zackay et al., “Detecting gravitational waves with disparate detector responses: two new binary black hole mergers”, [Phys. Rev. D](#) **104**, 063030 (2021).
- [14] T. Venumadhav et al., “New binary black hole mergers in the second observing run of Advanced LIGO and Advanced Virgo”, [Phys. Rev. D](#) **101**, 083030 (2020).
- [15] B. P. Abbott et al., “Characterization of transient noise in Advanced LIGO relevant to gravitational wave signal GW150914”, [Classical and Quantum Gravity](#) **33**, 134001 (2016).
- [16] B. P. Abbott et al., “A guide to ligo–virgo detector noise and extraction of transient gravitational-wave signals”, [Classical and Quantum Gravity](#) **37**, 055002 (2020).
- [17] D. Davis et al., “LIGO detector characterization in the second and third observing runs”, [Classical and Quantum Gravity](#) **38**, 135014 (2021).
- [18] S. Soni et al., “LIGO Detector Characterization in the first half of the fourth Observing run”, [Classical and Quantum Gravity](#) **42**, 085016 (2025).
- [19] B. P. Abbott et al., “Gw170817: observation of gravitational waves from a binary neutron star inspiral”, [Physical review letters](#) **119**, 161101 (2017).
- [20] B. P. Abbott et al., “Effects of data quality vetoes on a search for compact binary coalescences in Advanced LIGO’s first observing run”, [Classical and Quantum Gravity](#) **35**, 065010 (2018).
- [21] T. D. Canton, S. Bhagwat, S. V. Dhurandhar, and A. Lundgren, “Effect of sine-Gaussian glitches on searches for binary coalescence”, [Class. Quant. Grav.](#) **31**, 015016 (2014).
- [22] S. Ghonge et al., “Assessing and mitigating the impact of glitches on gravitational-wave parameter estimation: a model agnostic approach”, [Phys. Rev. D](#) **110**, 122002 (2024).

- [23] S. Hourihane et al., “Accurate modeling and mitigation of overlapping signals and glitches in gravitational-wave data”, *Phys. Rev. D* **106**, 042006 (2022).
- [24] R. Macas et al., “Impact of noise transients on low latency gravitational-wave event localization”, *Phys. Rev. D* **105**, 103021 (2022).
- [25] C. Pankow et al., “Mitigation of the instrumental noise transient in gravitational-wave data surrounding gw170817”, *Phys. Rev. D* **98**, 084016 (2018).
- [26] E. Payne et al., “Curious case of GW200129: Interplay between spin-precession inference and data-quality issues”, *Phys. Rev. D* **106**, 104017 (2022).
- [27] J. Powell, “Parameter estimation and model selection of gravitational wave signals contaminated by transient detector noise glitches”, *Classical and Quantum Gravity* **35**, 155017 (2018).
- [28] R. Udall et al., “Antialigned spin of GW191109: Glitch mitigation and its implications”, *Phys. Rev. D* **111**, 024046 (2025).
- [29] A. G. Abac et al. (LIGO Scientific, VIRGO, KAGRA), “GWTC-5.0: Methods for Identifying and Characterizing Gravitational-wave Transients”, (2026).
- [30] D. Davis et al., “Rapid data quality investigations of gravitational-wave events with the Data Quality Report Builder toolkit”, (2026).
- [31] L. Vazsonyi and D. Davis, “Identifying glitches near gravitational-wave signals from compact binary coalescences using the Q-transform”, *Class. Quant. Grav.* **40**, 035008 (2023).
- [32] S. Soni et al., “Reducing scattered light in LIGO’s third observing run”, *Classical and Quantum Gravity* **38**, 025016 (2021).
- [33] R. Biswas et al., “Application of machine learning algorithms to the study of noise artifacts in gravitational-wave data”, *Phys. Rev. D* **88**, 062003 (2013).
- [34] R. Essick, L. Blackburn, and E. Katsavounidis, “Optimizing Vetoes for Gravitational-Wave Transient Searches”, *Class. Quant. Grav.* **30**, 155010 (2013).
- [35] R. Essick et al., “iDQ: Statistical inference of non-gaussian noise with auxiliary degrees of freedom in gravitational-wave detectors”, *Machine Learning: Science and Technology* **2**, 015004 (2020).
- [36] S. Alvarez-Lopez et al., “GSPyNetTree: A signal-vs-glitch classifier for gravitational-wave event candidates”, *Classical and Quantum Gravity* (2024).
- [37] M. Zevin et al., “Gravity Spy: integrating advanced LIGO detector characterization, machine learning, and citizen science”, *Classical and Quantum Gravity* **34**, 064003 (2017).
- [38] M. Zevin et al., “Gravity Spy: lessons learned and a path forward”, *Eur. Phys. J. Plus* **139**, 100 (2024).
- [39] M. L. Chan et al., “GSPyNetTreeS: a machine learning solution for glitch localization in time and frequency”, (2025).
- [40] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: unified, real-time object detection”, in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016), pp. 779–788.
- [41] S. Bandyopadhyay et al., “GSPyNetTree-Bayes: Classification of Glitches and Gravitational Waves with Uncertainty”, in preparation.
- [42] A. G. Abac et al. (LIGO Scientific, VIRGO, KAGRA), “GWTC-5.0: Observations from the Second Part of the Fourth LIGO-Virgo-KAGRA Observing Run and Updates to the Gravitational-Wave Transient Catalog”, (2026).
- [43] C. Szegedy et al., “Rethinking the Inception Architecture for Computer Vision”, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 2818–2826.
- [44] J. Glanzer et al., “Data quality up to the third observing run of advanced LIGO: Gravity Spy glitch classifications”, *Classical and Quantum Gravity* **40**, 065004 (2023).
- [45] S. Soni et al., “Discovering features in gravitational-wave data through detector characterization, citizen science, and machine learning”, *Classical and Quantum Gravity* **38**, 195016 (2021).

- [46] Y. Wu et al., “Advancing glitch classification in gravity spy: multi-view fusion with attention-based machine learning for advanced ligo’s fourth observing run”, *Classical and Quantum Gravity* **42**, 165015 (2025).
- [47] S. Jarov et al., “A new method to distinguish gravitational-wave signals from detector noise transients with gravity spy”, arXiv preprint arXiv:2307.15867 (2023).
- [48] Pace A, Prestegard T, Moe B and Stephens B, “GraceDB—Gravitational-Wave Candidate Event Database”, <https://gracedb.ligo.org/> (2020).
- [49] S. A. Usman et al., “The PyCBC search for gravitational waves from compact binary coalescence”, *Classical and Quantum Gravity* **33**, 215004 (2016).
- [50] A. H. Nitz et al., “Detecting binary compact-object mergers with gravitational waves: understanding and improving the sensitivity of the pycbc search”, *The Astrophysical Journal* **849**, 118 (2017).
- [51] T. Adams et al., “Low-latency analysis pipeline for compact binary coalescences in the advanced gravitational wave detector era”, *Classical and Quantum Gravity* **33**, 175012 (2016).
- [52] F. Aubin et al., “The MBTA pipeline for detecting compact binary coalescences in the third LIGO–Virgo observing run”, *Classical and Quantum Gravity* **38**, 095004 (2021).
- [53] C. Messick et al., “Analysis framework for the prompt discovery of compact binary mergers in gravitational-wave data”, *Phys. Rev. D* **95**, 042001 (2017).
- [54] S. Sachdev et al., “The GstLAL Search Analysis Methods for Compact Binary Mergers in Advanced LIGO’s Second and Advanced Virgo’s First Observing Runs”, (2019).
- [55] C. Hanna et al., “Fast evaluation of multidetector consistency for real-time gravitational wave searches”, *Physical Review D* **101**, 10.1103/physrevd.101.022003 (2020).
- [56] K. Cannon et al., “GstLAL: A software framework for gravitational wave discovery”, 2020.
- [57] J. Luan, S. Hooper, L. Wen, and Y. Chen, “Towards low-latency real-time detection of gravitational waves from compact binary coalescences in the era of advanced detectors”, *Phys. Rev. D* **85**, 102002 (2012).
- [58] Q. Chu et al., “SPIIR online coherent pipeline to search for gravitational waves from compact binary coalescences”, *Phys. Rev. D* **105**, 024023 (2022).
- [59] J. S. Areeda et al., *LigoDV-web*, arXiv:1611.01089 [astro-ph, physics:gr-qc], Nov. 2016.
- [60] LIGO Scientific Collaboration, Virgo Collaboration and KAGRA Collaboration, “GWTC-3 Data Release”, <https://www.gw-openscience.org/GWTC-3/> (2021).
- [61] R. Abbott et al., “Open data from the first and second observing runs of advanced ligo and advanced virgo”, *SoftwareX* **13**, 100658 (2021).
- [62] R. Abbott et al., “Open data from the third observing run of ligo, virgo, kagra and geo”, arXiv preprint arXiv:2302.03676 (2023).
- [63] D. Davis et al., “Subtracting glitches from gravitational-wave detector data during the third LIGO-virgo observing run”, *Classical and Quantum Gravity* **39**, 245013 (2022).
- [64] The LIGO Scientific Collaboration, *LIGO Algorithm Library - LALSuite*, free software (GPL), 2018.
- [65] S. Husa et al., “Frequency-domain gravitational waves from nonprecessing black-hole binaries. I. New numerical waveforms and anatomy of the signal”, *Phys. Rev. D* **93**, 044006 (2016).
- [66] S. Khan et al., “Frequency-domain gravitational waves from nonprecessing black-hole binaries. II. A phenomenological model for the advanced detector era”, *Phys. Rev. D* **93**, 044007 (2016).
- [67] R. P. Udall and D. Davis, “Bayesian modeling of scattered light in the LIGO interferometers”, *Applied Physics Letters* **122**, 094103 (2023).
- [68] R. Bondarescu, A. Lundgren, and R. Macas, “Quasiphsical model for removing short glitches from LIGO and Virgo data”, *Phys. Rev. D* **108**, 122004 (2023).
- [69] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (O’Reilly Media, Inc., 2022).
- [70] R. Meudec, *tf-explain*, comp. software, version 0.3.1, Interpretability Methods for tf.keras models with TensorFlow 2.x, Feb. 4, 2021.

- [71] R. R. Selvaraju et al., “Grad-cam: visual explanations from deep networks via gradient-based localization”, in Proceedings of the IEEE International Conference on Computer Vision (2017), pp. 618–626.
- [72] [GCN Notice for S230708bi](#), GCN Notice, 2023.
- [73] LIGO Scientific Collaboration, Virgo Collaboration and KAGRA Collaboration, “LIGO/Virgo/KAGRA S230708bi: Retraction of GW compact binary merger candidate”, GRB Coordinates Network **34172** (2023).
- [74] S. Chatterji, L. Blackburn, G. Martin, and E. Katsavounidis, “Multiresolution techniques for the detection of gravitational-wave bursts”, [Classical and Quantum Gravity](#) **21**, S1809 (2004).
- [75] T. A. Ferreira and G. González, “Using t-SNE for characterizing glitches in LIGO detectors”, [Class. Quant. Grav.](#) **42**, 105010 (2025).
- [76] T. Fernandes et al., “VIGILant: an automatic classification pipeline for glitches in the Virgo detector”, (2026).